

```

timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"

module tb_EX;

    // Inputs
    reg clk;
    reg rst_n;
    reg [4:0] a1, a2;
    reg [31:0] i_rs1_data, i_rs2_data, i_Pc, i_Imm;
    reg [3:0] i_AlucCtrl;
    reg [2:0] i_Func3;
    reg [4:0] i_Rd;
    reg i_RegSrc, i_Sel1, i_Sel2, i_Load, i_Branch, i_Jal, i_Jalr, i_MemSrc,
i_is_amo;
    reg [1:0] i_ResultSrc;
    reg [4:0] i_amo_func5;
    reg [1:0] i_mux1_sel, i_mux2_sel;
    reg [31:0] i_ex_result, i_w_data;

    // Outputs
    wire [31:0] o_DataStoreM, o_ResultM;
    wire [2:0] o_Func3M;
    wire [4:0] o_RdM;
    wire o_RegSrcM, o_MemSrcM, o_LoadM, o_Boj, o_Jalr;
    wire [1:0] o_ResultSrcM;
    wire [31:0] o_PcM, o_Result, o_Imm_data;
    wire [31:0] fwdwire1, fwdwire2;

    // Instantiate the EX module
    EX ex_inst (
        .clk(clk),
        .rst_n(rst_n),
        .a1(a1),
        .a2(a2),
        .i_rs1_data(i_rs1_data),
        .i_rs2_data(i_rs2_data),
        .i_Pc(i_Pc),
        .i_AlucCtrl(i_AlucCtrl),
        .i_Func3(i_Func3),
        .i_Rd(i_Rd),
        .i_RegSrc(i_RegSrc),
        .i_Imm(i_Imm),
        .i_Sel1(i_Sel1),
        .i_Sel2(i_Sel2),

```

```

.i_Load(i_Load),
.i_Branch(i_Branch),
.i_Jal(i_Jal),
.i_Jalr(i_Jalr),
.i_MemSrc(i_MemSrc),
.i_ResultSrc(i_ResultSrc),
.i_amo_funct5(i_amo_funct5),
.i_is_amo(i_is_amo),
.o_DataStoreM(o_DataStoreM),
.o_ResultM(o_ResultM),
.o_Func3M(o_Func3M),
.o_RdM(o_RdM),
.o_RegSrcM(o_RegSrcM),
.o_MemSrcM(o_MemSrcM),
.o_ResultSrcM(o_ResultSrcM),
.o_PcM(o_PcM),
.o_LoadM(o_LoadM),
.o_Result(o_Result),
.o_Boj(o_Boj),
.o_Jalr(o_Jalr),
.o_Imm_data(o_Imm_data),
.fwdwire1(fwdwire1),
.fwdwire2(fwdwire2),
.i_mux1_sel(i_mux1_sel),
.i_mux2_sel(i_mux2_sel),
.i_ex_result(i_ex_result),
.i_w_data(i_w_data)
);

// Clock generator
always #5 clk = ~clk;

initial begin
    $display("Starting EX Testbench...");
    $dumpfile("tb_EX.vcd");
    $dumpvars(0, tb_EX);

    // Init signals
    clk = 0;
    rst_n = 0;
    a1 = 5'b0;
    a2 = 5'b0;
    i_rs1_data = 32'h00000005;
    i_rs2_data = 32'h00000003;
    i_Pc = 32'h00000000;
    i_Aluctrl = 4'b0000;

```

```

i_Func3 = 3'b000;
i_Rd = 5'd1;
i_RegSrc = 0;
i_Imm = 32'h00000000;
i_Sel1 = 1;
i_Sel2 = 1;
i_Load = 0;
i_Branch = 0;
i_Jal = 0;
i_Jalr = 0;
i_MemSrc = 0;
i_ResultSrc = 2'b00;
i_mux1_sel = 2'b00;
i_mux2_sel = 2'b00;
i_ex_result = 32'b0;
i_w_data = 32'b0;
i_is_amo = 0;
i_amo_func5 = 5'b00000;

// Reset pulse
#10 rst_n = 1;

// -----
// Test 1: AMOADD
// -----
i_is_amo = 1;
i_amo_func5 = `AMO_ADD;
#10;
$display("AMOADD Result = %0d", o_Result); // Expect 8

// -----
// Test 2: AMOSWAP
// -----
i_amo_func5 = `AMO_SWAP;
#10;
$display("AMOSWAP Result = %0d", o_Result); // Expect 3 (i_rs2_data)

// -----
// Test 3: AMOAND
// -----
i_rs1_data = 32'hF0F0F0F0;
i_rs2_data = 32'h0F0F0F0F;
i_amo_func5 = `AMO_AND;
#10;
$display("AMOAND Result = %h", o_Result); // Expect 00000000

```

```

// -----
// Test 4: AMOMAX
// -----
i_rs1_data = 32'h00000009;
i_rs2_data = 32'h00000005;
i_amo_func5 = `AMO_MAX;
#10;
$display("AMOMAX Result = %0d", o_Result); // Expect 9
// Test AMO_LR
#10;
i_amo_func5 = `AMO_LR;
i_rs1_data = 32'hCAFEFEBABE;
i_rs2_data = 32'hDEADBEEF;
i_is_amo = 1;

// Test AMO_SC
#10;
$display("AMOMAX Result = %0d", o_Result); // Expect 9

i_amo_func5 = `AMO_SC;
i_rs1_data = 32'hFFFFFFFF;
i_rs2_data = 32'hA5A5A5A5;
i_is_amo = 1;
#10;
$display("AMOMAX Result = %0d", o_Result); // Expect 9

// End simulation
#10;
$finish;
end

endmodule

```